XCONFIGURE

XCONFIGURE is a collection of configure wrapper scripts for various HPC applications. The purpose of the scripts is to configure the application in question with optimized flags for the system running the script, primarily using the GNU Compiler Collection (`configure-<name>.sh`) and alternatively the Intel Compiler (`configure-<name>-icx.sh`). XCONFIGURE helps to rely on a "build recipe", which is known to expose the highest performance or to reliably complete the build process.

> Contributions are very welcome!

Each application (or library) is hosted in a separate directory. To configure (and ultimately build) an application, one can rely on a single script which then downloads a specific wrapper into the current working directory (of the desired application).

```
wget --no-check-certificate https://github.com/hfp/xconfigure/raw/main/configure-get.sh
chmod +x configure-get.sh

echo "EXAMPLE: recipe for LIBXC"
./configure-get.sh libxc
```

The scripts generate code for the system running the script (`-march=native`, `-mcpu=native` on Arm, or `-xHost`). Libraries are installed next to the source code using `<name>/<arch>/<tagname>` (e.g., `libint/sapphirerapids/gnu`), and CMake-based builds use `build-<arch>-<tagname>` inside of the source folder. The `<arch>` is determined by the compiler (e.g., `sapphirerapids` or `neoverse-v2`), which allows to share one folder structure (NFS) among different systems, and `XCFG_ARCH` overrides the name.

On systems without access to the Internet, one can download (or clone) the entire collection upfront. To configure an application, please open the config folder directly or use the documentation and then follow the build recipe of the desired application or library.

Documentation

- **ReadtheDocs**: online documentation with full text search: CP2K, ELPA, LIBINT, LIBXC, LIBWIGNERNJ, LIBVORI, FFTW, DBCSR, and LIBXSMM.
- **PDF**: a single documentation file.

Related Projects

- Spack Package Manager: [http://computation.llnl.gov/projects/spack-hpc-package-manager%5D(http://computation.llnl.gov/projects/spack-hpc-package-manager)
- EasyBuild / EasyConfig (University of Gent): [https://github.com/easybuilders%5D(https://github.com/easybuilders)

Please note that XCONFIGURE has a narrower scope when compared to the above package managers.

Applications

CP2K

This document describes building CP2K with CMake and with dependencies built by XCONFIGURE, with a focus on GPU support by the means of OpenCL (LIBXSTREAM). All packages are placed into a common directory (`$HOME` is assumed), and the dependencies are installed next to CP2K, where its configure script finds them.

Package	Needed	Source	Inst

Dependencies

Package	Needed	Source	Inst
BLAS, LAPACK, ScaLAPACK	required	MKL, NVPL, or OpenBLAS+ScaLAPACK	syst
FFTW3	required	MKL, NVPL, or FFTW	`fftw`
DBCSR	required	GitHub	`dbcs`
LIBWIGNERNJ	required	cp2k.org	`libw`
LIBINT	strongly recommended	cp2k.org (preconfigured)	`libi`
LIBXC	strongly recommended	cp2k.org	`libx`
LIBVORI	optional	cp2k.org	`libv`
LIBXSTREAM	GPU support (OpenCL)	GitHub	`libx`
LIBXS	for LIBXSTREAM, recommended	GitHub	`libx`
LIBXSMM	optional (with LIBXS)	GitHub	`libx`

The sources of the dependencies are best taken from CP2K's download area, which carries the versions CP2K is tested with. LIBINT in particular must be one of CP2K's preconfigured packages (`libint-v<version>-cp2k-lmax-<lmax>.tar.xz`), since LIBINT's own source code is a generator rather than the library. LIBINT and LIBXC are optional for CP2K but needed to run common workloads: inputs using Hartree-Fock exchange (e.g., hybrid functionals) require LIBINT, and many exchange-correlation functionals require LIBXC. LIBXS is required for GPU support by LIBXSTREAM, and is useful for CPU-only builds as well (small matrix multiplications, timers, memory pool).

Further prerequisites are the GNU Compiler Collection, MPI (compiler wrappers `mpicc`, `mpicxx`, and `mpif90`), CMake, and Python. Intel MKL replaces OpenBLAS, ScaLAPACK, and FFTW at once.

Step-by-step Guide (GPU/OpenCL)

The prerequisites can be installed using the package manager, e.g., on Debian or Ubuntu:

```
sudo apt install build-essential gfortran cmake python3 git wget pkg-config \
  libopenmpi-dev openmpi-bin libopenblas-openmp-dev libscalapack-openmpi-dev \
  opencl-headers ocl-icd-opencl-dev clinfo
```

OpenCL consists of headers, the dispatch library (ICD loader `libOpenCL.so`), and at least one driver registered in `/etc/OpenCL/vendors`. The above `opencl-headers` and `ocl-icd-opencl-dev` packages provide the headers and the dispatch library. The driver is part of the NVIDIA driver (e.g., `libnvidia-compute-<version>`), or `intel-opencl-icd` for Intel GPUs (Intel Compute Runtime). CUDA (`CUDA_PATH` or `nvcc` in `PATH`) and Intel oneAPI (`CMPLR_ROOT`) also provide headers and a dispatch library, and are preferred in this order when found (oneAPI's dispatch library depends on Intel runtime libraries, and sourcing oneAPI is not needed for building with the GNU Compiler Collection). The available devices are listed by `clinfo -l`.

An older OpenBLAS (e.g., 0.3.20) may not know a newer CPU and compute wrong results with the kernels it selects for it, which `OPENBLAS_CORETYPE=Haswell` avoids at runtime (or a newer OpenBLAS). GPU-aware MPI is not needed; for Intel GPUs, it is not packaged and requires building MPICH with Level Zero support (`--with-ze`).

All packages are placed into `$HOME`, where each step finds the previous ones:

```
cd $HOME
wget --content-disposition https://github.com/hfp/xconfigure/raw/main/configure-get.sh
chmod +x configure-get.sh

git clone https://github.com/hfp/libxs.git && make -C libxs -j $(nproc)
git clone https://github.com/hfp/libxstream.git && make -C libxstream -j $(nproc)

git clone --recursive https://github.com/cp2k/dbcsr.git && cd dbcsr
../configure-get.sh dbcsr && ./configure-dbcsr-gpu.sh
cmake --build build-*-gnu-gpu --target install -j $(nproc)
cd $HOME

wget https://www.cp2k.org/static/downloads/libwignernj-0.8.0.tar.gz
```

```
tar xf libwignernj-0.8.0.tar.gz && cd libwignernj-0.8.0
../configure-get.sh libwignernj && ./configure-libwignernj.sh
cmake --build build-*-gnu --target install -j $(nproc)
cd $HOME

wget https://www.cp2k.org/static/downloads/libxc-7.1.2.tar.bz2
tar xf libxc-7.1.2.tar.bz2 && cd libxc-7.1.2
../configure-get.sh libxc && ./configure-libxc.sh
cmake --build build-*-gnu --target install -j $(nproc)
cd $HOME

wget https://www.cp2k.org/static/downloads/libint-v2.13.1-cp2k-lmax-6.tar.xz
tar xf libint-v2.13.1-cp2k-lmax-6.tar.xz && cd libint-v2.13.1-cp2k-lmax-6
../configure-get.sh libint && ./configure-libint.sh
cmake --build build-*-gnu --target install -j $(nproc)
cd $HOME

wget https://www.fftw.org/fftw-3.3.11.tar.gz
tar xf fftw-3.3.11.tar.gz && cd fftw-3.3.11
../configure-get.sh fftw && ./configure-fftw.sh
make -C build-*-gnu install -j $(nproc)
cd $HOME

git clone https://github.com/cp2k/cp2k.git && cd cp2k
../configure-get.sh cp2k && ./configure-cp2k-gpu.sh
cmake --build build-*-gnu-gpu -j $(nproc)
./run.sh benchmarks/QS/H2O-64.inp
```

The `build-*` folders assume a single architecture per source folder (see below); the configure scripts print the exact build command. Building LIBINT takes the longest (several minutes). FFTW is not needed with Intel MKL (`MKLROOT`), which also replaces OpenBLAS and ScaLAPACK, or with NVPL (e.g., NVIDIA Grace), which provides FFTW's interface. To rely on NVPL's BLAS, LAPACK, and ScaLAPACK instead of OpenBLAS, `NVPL=1` must be used for DBCSR and CP2K alike (`NVPL=1 ./configure-dbcsr-gpu.sh` and `NVPL=1 ./configure-cp2k-gpu.sh`).

Build Instructions

```
cd $HOME && git clone https://github.com/cp2k/cp2k.git
cd cp2k

wget --content-disposition https://github.com/hfp/xconfigure/raw/main/configure-get.sh
chmod +x configure-get.sh
./configure-get.sh cp2k
```

To configure and build CP2K (`build-<arch>-gnu-cpu`), or CP2K with GPU support by the means of OpenCL (`build-<arch>-gnu-gpu`):

```
./configure-cp2k.sh
cmake --build build-<arch>-gnu-cpu -j $(nproc)

./configure-cp2k-gpu.sh
cmake --build build-<arch>-gnu-gpu -j $(nproc)
```

CP2K can be run from where it was built, e.g., `build-<arch>-gnu-cpu/bin/cp2k.psmp` (see "Running CP2K" below). Optionally, `--target install` installs into `../cp2k/<arch>/gnu-cpu` (`../cp2k/<arch>/gnu-gpu`).

Here, `<arch>` names the target as determined by the compiler (e.g., `sapphirerapids` or `neoverse-v2`), which allows to share the folder structure among different systems (`XCFG_ARCH` overrides the name). For each dependency, the first of `../<name>/<arch>/gnu-cpu` (`gnu-gpu`), `../<name>/<arch>/gnu-cpu-shared` (`gnu-gpu-shared`), `../<name>/<arch>/gnu`, `../<name>/<arch>`, the same folders without `<arch>`, `$<NAME>ROOT` (e.g., `LIBXSROOT`), and the same folders in `$HOME` containing a library or package is used, and the corresponding feature of CP2K is enabled. For example, `dbcsr/<arch>/gnu` can be a symbolic link to the desired flavor of DBCSR. The found locations are printed when configuring CP2K, and a DBCSR that was built but not installed is pointed out. If built with GNU Make, the matching `lib/<target>` of LIBXS, LIBXSMM, or LIBXSTREAM is used (e.g., `libxs/lib/x86_64`). Further dependencies can be supplied using the `CMAKE_PREFIX_PATH` environment variable, and arguments after an optional "tagname" (default: `gnu`) are passed to CMake:

```
CMAKE_PREFIX_PATH=$HOME/elpa/gnu ./configure-cp2k.sh gnu -DCP2K_USE_ELPA=ON
```

If NVPL is installed (e.g., on NVIDIA Grace, `NVPL_ROOT` if not in a system location), its FFTW interface is used unless FFTW was built by XCONFIGURE. With `NVPL=1`, CP2K also relies on NVPL's BLAS, LAPACK, and ScaLAPACK; DBCSR must be configured with `NVPL=1` as well to link the same BLAS library:

```
NVPL=1 ./configure-cp2k-gpu.sh
```

Code is generated for the system running the script (`-march=native`, or `-mcpu=native` on Arm), i.e., a build system matching the target system is assumed.

CP2K on GPUs

The OpenCL support enables DBCSR's OpenCL backend as well as CP2K's GPU-enabled DBM/DBT component. The OpenCL backend has comprehensive runtime-control by the means of environment variables. This can be used to assign OpenCL devices, to aggregate sub-devices (devices are split into sub-devices by default), to extract kernel shapes used by a specific workload, and to subsequently tune specific kernels. DBCSR as well as CP2K's DBM component (`src/dbm`) can be exercised in a standalone fashion. The LIBXSTREAM documentation describes the runtime settings of the OpenCL backend (e.g., device selection and profiling), and its OpenCL-MPS lets several ranks per node share one context per device.

Running CP2K

Running CP2K may go beyond a single node, and pinning processes and threads become even more important. There are several schemes available. As a rule of thumb, a high rank-count for lower node-counts may yield best results unless the workload is very memory intensive. In the latter case, lowering the number of MPI-ranks per node is effective especially if a larger amount of memory is replicated rather than partitioned by the rank-count. In contrast (communication bound), a lower rank count for multi-node computations may be desired. To ease running CP2K, there are a number of supportive scripts provided by XCONFIGURE: `plan.sh` (see here), `run.sh` (see here), and `info.sh`.

The run-script can be symlinked into CP2K's root directory (`ln -s /path/to/xconfigure/config/cp2k/run.sh`). Unless `EXE` names the executable, the run-script looks for `cp2k.psmp` (`CP2K_VERSION=ssmp` for `cp2k.ssmp`) in the following order:

- `exe/Linux-x86-64-intelx` (legacy build, `CP2K_BUILD` selects another ARCH)
- `build/bin` relative to the current directory (legacy location)
- `<CP2K_BUILD>/bin` or `build-<arch>-<CP2K_BUILD>*/bin` if `CP2K_BUILD` is given (e.g., `CP2K_BUILD=build-alderlake-gnu-cpu` or `CP2K_BUILD=gnu-gpu`), where `<arch>` is determined like for building
- otherwise `build/bin` in CP2K's root directory (e.g., a symbolic link), the most recent `build-<arch>-*/bin`, or `build-*/bin` if there is only one
- `../cp2k/<arch>/gnu/bin` and `../cp2k/gnu/bin` (installation, `CP2K_BUILD` selects another tagname)

The run-script's options and environment variables are shown by `./run.sh --help`, which also prints the selected executable.

As soon as several experiments are finished, it becomes handy to summarize the log-output. For this case, an info-script (`info.sh`) is available attempting to present a table (summary of all results), which is generated from log files (`.txt` and `.out` extension by default). Log files can be captured with "tee", or the output is captured by the job scheduler.

```
./run.sh benchmarks/QS/H2O-64.inp | tee cp2k-h2o64-20240725b.txt
ls -1 *.txt
cp2k-h2o64-20240725a.txt
cp2k-h2o64-20240725b.txt

./info.sh [-best] [/path/to/logs]
H2O-64                Nodes R/N T/R Cases/d Seconds
cp2k-h2o64-20240725a 2       32   4     807 107.237
cp2k-h2o64-20240725b 4       16   8     872  99.962
```

Note: the "*Cases/d*" metric is calculated with integer arithmetic and hence represents fully completed cases per day (based on 86400 seconds per day). The number of seconds (as shown) is end-to-end (wall time), i.e., total time to solution including any (sequential) phase (initialization, etc.). Performance is higher if the workload requires more iterations (some publications present a metric based on iteration time).

Sanity Check

There is nothing that can replace the full regression test suite. However, to quickly check whether a build is sane or not, one can run for instance `benchmarks/QS/H2O-64.inp` and check if the SCF iteration prints like the following:

```
 Step     Update method      Time    Convergence         Total energy    Change
 -------------------------------------------------------------------------------
    1 OT DIIS     0.15E+00    0.5     0.01337191     -1059.6804814927 -1.06E+03
    2 OT DIIS     0.15E+00    0.3     0.00866338     -1073.3635678409 -1.37E+01
    3 OT DIIS     0.15E+00    0.3     0.00615351     -1082.2282197787 -8.86E+00
    4 OT DIIS     0.15E+00    0.3     0.00431587     -1088.6720379505 -6.44E+00
    5 OT DIIS     0.15E+00    0.3     0.00329037     -1092.3459788564 -3.67E+00
    6 OT DIIS     0.15E+00    0.3     0.00250764     -1095.1407783214 -2.79E+00
    7 OT DIIS     0.15E+00    0.3     0.00187043     -1097.2047924571 -2.06E+00
    8 OT DIIS     0.15E+00    0.3     0.00144439     -1098.4309205383 -1.23E+00
    9 OT DIIS     0.15E+00    0.3     0.00112474     -1099.2105625375 -7.80E-01
   10 OT DIIS     0.15E+00    0.3     0.00101434     -1099.5709299131 -3.60E-01
   [...]
```

The column called "*Convergence*" must monotonically converge towards zero.

References

[https://nholmber.github.io/2017/04/cp2k-build-cray-xc40/%5D(https://nholmber.github.io/2017/04/cp2k-build-cray-xc40/)
[https://xconfigure.readthedocs.io/cp2k/plan/%5D(https://xconfigure.readthedocs.io/cp2k/plan/)
[https://www.cp2k.org/static/downloads%5D(https://www.cp2k.org/static/downloads)
[https://www.cp2k.org/howto:compile%5D(https://www.cp2k.org/howto:compile)

DBCSR

To configure, build, and install DBCSR for CP2K, one may proceed as shown below.

```
git clone --recursive https://github.com/cp2k/dbcsr.git
cd dbcsr

wget --content-disposition https://github.com/hfp/xconfigure/raw/main/configure-get.sh
chmod +x configure-get.sh
./configure-get.sh dbcsr
```

There are four flavors, which differ by the accelerator backend ("cpu" or "gpu") and by the kind of library (static or "shared"). The build relies on the GNU Compiler Collection and on MPI compiler wrappers (`mpicc`, `mpicxx`, `mpif90`). LIBXS is used for host-side small matrix multiplications, and the GPU flavors rely on OpenCL and LIBXSTREAM. LIBXS, LIBXSMM (optional), and LIBXSTREAM are looked up next to DBCSR (e.g., `../libxs`), via `LIBXSROOT` (`LIBXSMMROOT`, `LIBXSTREAMROOT`), and in `$HOME` (e.g., `$HOME/libxs`), and the found locations are printed. If built with GNU Make, the matching `lib/<target>` of LIBXS, LIBXSMM, or LIBXSTREAM is used (e.g., `libxs/lib/x86_64`). LIBXSMM is enabled if found. Other locations can be supplied using the `CMAKE_PREFIX_PATH` environment variable.

```
./configure-dbcsr-gpu-shared.sh
cmake --build build-<arch>-gnu-gpu-shared --target install -j $(nproc)
```

Here, `<arch>` names the target as determined by the compiler (e.g., `sapphirerapids` or `neoverse-v2`), which allows to share the folder structure among different systems (`XCFG_ARCH` overrides the name). The above installs into `../dbcsr/<arch>/gnu-gpu-shared`, and similarly for `configure-dbcsr-cpu.sh` (`gnu-cpu`), `configure-dbcsr-cpu-shared.sh` (`gnu-cpu-shared`), and `configure-dbcsr-gpu.sh` (`gnu-gpu`). An optional first argument changes the "gnu" part of the name, and further arguments are passed to CMake, e.g., to select another GPU backend:

```
./configure-dbcsr-gpu.sh gnu -DUSE_ACCEL=cuda -DWITH_GPU=H100
```

BLAS and LAPACK are found by CMake (e.g., OpenBLAS or Intel MKL). With `NVPL=1`, NVPL's BLAS and LAPACK are used instead (e.g., on NVIDIA Grace, `NVPL_ROOT` if not in a system location), and CP2K must be configured with `NVPL=1` as well to link the same BLAS library.

References

[https://github.com/cp2k/dbcsr%5D(https://github.com/cp2k/dbcsr)
[https://cp2k.github.io/dbcsr/%5D(https://cp2k.github.io/dbcsr/)

ELPA

Download and unpack ELPA and make the configure wrapper scripts available in ELPA's root folder. Consider CP2K's download area (cache) as an alternative source for downloading ELPA.

```
echo "wget --content-disposition https://www.cp2k.org/static/downloads/elpa-2021.11.002.tar.gz"
wget --content-disposition https://elpa.mpcdf.mpg.de/software/tarball-archive/Releases/2021.11.002/elp
tar xvf elpa-2021.11.002.tar.gz
cd elpa-2021.11.002

wget --content-disposition https://github.com/hfp/xconfigure/raw/main/configure-get.sh
chmod +x configure-get.sh
./configure-get.sh elpa
```

The scripts rely on Intel MKL (`MKLROOT`), and on the GNU Compiler Collection (`configure-elpa.sh`) or the Intel Compiler (`configure-elpa-icx.sh`). Code is generated for the system running the script. To configure, build, and install (`../elpa/<arch>/gnu`):

```
make clean
./configure-elpa.sh
make -j ; make install
```

Even if ELPA was just unpacked (and never built before), `make clean` is recommended in advance of building ELPA ("unknown module file format"). OpenMP is enabled by default, and `OMP=0` disables it. An optional first argument ("tagname") changes the name of the installation folder:

```
make clean
OMP=0 ./configure-elpa.sh gnu-noomp
make -j ; make install
```

Here, `<arch>` names the target as determined by the compiler (e.g., `sapphirerapids` or `neoverse-v2`), which allows to share the folder structure among different systems (`XCFG_ARCH` overrides the name).

References

[https://github.com/cp2k/cp2k/blob/master/INSTALL.md#2l-elpa-optional-improved-performance-for-diagonalization%5D(https://github.com/cp2k/cp2k/blob/master/INSTALL.md#2l-elpa-optional-improved-performance-for-diagonalization)
[https://elpa.mpcdf.mpg.de/software/tarball-archive/ELPA_TARBALL_ARCHIVE.html%5D(https://elpa.mpcdf.mpg.de/software/tarball-archive/ELPA_TARBALL_ARCHIVE.html)
[https://www.cp2k.org/static/downloads%5D(https://www.cp2k.org/static/downloads)

FFTW

To download, configure, build, and install FFTW (double precision with OpenMP support as needed by CP2K), one may proceed as shown below.

```
wget https://www.fftw.org/fftw-3.3.11.tar.gz
tar xvf fftw-3.3.11.tar.gz
cd fftw-3.3.11

wget --content-disposition https://github.com/hfp/xconfigure/raw/main/configure-get.sh
chmod +x configure-get.sh
./configure-get.sh fftw
```

To configure, build, and install using the GNU Compiler Collection (`../fftw/<arch>/gnu`):

```
./configure-fftw.sh
make -C build-<arch>-gnu install -j $(nproc)
```

To use the Intel Compiler instead (`../fftw/<arch>/icx`):

```
./configure-fftw-icx.sh
make -C build-<arch>-icx install -j $(nproc)
```

Here, `<arch>` names the target as determined by the compiler (e.g., `sapphirerapids` or `neoverse-v2`), which allows to share the folder structure among different systems (`XCFG_ARCH` overrides the name). SIMD support is enabled according to the system running the script (SSE2, AVX, AVX2, AVX-512, or NEON on Arm). An optional first argument ("tagname") changes the name of the build and installation folder. Further arguments are passed to FFTW's configure script.

FFTW is not needed if CP2K relies on Intel MKL, or if NVPL provides FFTW's interface (e.g., on NVIDIA Grace).

References

[https://www.fftw.org/%5D(https://www.fftw.org/)
[https://www.cp2k.org/static/downloads/%5D(https://www.cp2k.org/static/downloads/)

LIBINT

LIBINT is a generator for integral code, and CP2K requires a library which is already generated with CP2K's settings (angular momentum, derivative order, etc.). Such preconfigured source code is available from CP2K's download area (`libint-v<version>-cp2k-lmax-<lmax>.tar.xz`, take "lmax-6" if unsure). XCONFIGURE does not support generic LIBINT source code (that is, generating the library).

```
wget https://www.cp2k.org/static/downloads/libint-v2.13.1-cp2k-lmax-6.tar.xz
tar xvf libint-v2.13.1-cp2k-lmax-6.tar.xz
cd libint-v2.13.1-cp2k-lmax-6

wget --content-disposition https://github.com/hfp/xconfigure/raw/main/configure-get.sh
chmod +x configure-get.sh
./configure-get.sh libint
```

To configure, build, and install using the GNU Compiler Collection (`../libint/<arch>/gnu`):

```
./configure-libint.sh
cmake --build build-<arch>-gnu --target install -j $(nproc)
```

To use the Intel Compiler instead (`../libint/<arch>/icx`):

```
./configure-libint-icx.sh
cmake --build build-<arch>-icx --target install -j $(nproc)
```

Here, `<arch>` names the target as determined by the compiler (e.g., `sapphirerapids` or `neoverse-v2`), which allows to share the folder structure among different systems (`XCFG_ARCH` overrides the name). The build relies on CMake and Python (Fortran interface). Neither Boost nor Eigen is necessary since the C++ API is not built. Code is generated for the system running the script (`-march=native`, `-mcpu=native` on Arm, or `-xHost`), i.e., a build system matching the target system is assumed.

An optional first argument ("tagname") changes the name of the build and installation folder, e.g., `./configure-libint.sh gnu-t` installs into `../libint/<arch>/gnu-test`. Further arguments are passed to CMake.

References

[https://www.cp2k.org/static/downloads/%5D(https://www.cp2k.org/static/downloads/)
[https://github.com/evaleev/libint%5D(https://github.com/evaleev/libint)

LIBVORI

libvori provides Voronoi integration and the BQB format (lossless compression of volumetric data) to CP2K, which is optional (`CP2K_USE_VORI`). To download, configure, build, and install, one may proceed as shown below.

```
wget https://www.cp2k.org/static/downloads/libvori-220621.tar.gz
tar xvf libvori-220621.tar.gz
cd libvori-220621

wget --content-disposition https://github.com/hfp/xconfigure/raw/main/configure-get.sh
chmod +x configure-get.sh
./configure-get.sh libvori
```

To configure, build, and install using the GNU Compiler Collection (`../libvori/<arch>/gnu`):

```
./configure-libvori.sh
cmake --build build-<arch>-gnu --target install -j $(nproc)
```

To use the Intel Compiler instead (`../libvori/<arch>/icx`):

```
./configure-libvori-icx.sh
cmake --build build-<arch>-icx --target install -j $(nproc)
```

Here, `<arch>` names the target as determined by the compiler (e.g., `sapphirerapids` or `neoverse-v2`), which allows to share the folder structure among different systems (`XCFG_ARCH` overrides the name). An optional first argument ("tagname") changes the name of the build and installation folder. Further arguments are passed to CMake. When configuring CP2K, `configure-cp2k.sh` finds the installation and enables libvori.

References

[https://brehm-research.de/libvori.php%5D(https://brehm-research.de/libvori.php)
[https://www.cp2k.org/static/downloads/%5D(https://www.cp2k.org/static/downloads/)

LIBWIGNERNJ

CP2K requires libwignernj (version 0.8 or later). To download, configure, build, and install, one may proceed as shown below.

```
wget https://www.cp2k.org/static/downloads/libwignernj-0.8.0.tar.gz
tar xvf libwignernj-0.8.0.tar.gz
cd libwignernj-0.8.0

wget --content-disposition https://github.com/hfp/xconfigure/raw/main/configure-get.sh
chmod +x configure-get.sh
./configure-get.sh libwignernj
```

To configure, build, and install using the GNU Compiler Collection (`../libwignernj/<arch>/gnu`):

```
./configure-libwignernj.sh
cmake --build build-<arch>-gnu --target install -j $(nproc)
```

To use the Intel Compiler instead (`../libwignernj/<arch>/icx`):

```
./configure-libwignernj-icx.sh
cmake --build build-<arch>-icx --target install -j $(nproc)
```

Here, `<arch>` names the target as determined by the compiler (e.g., `sapphirerapids` or `neoverse-v2`), which allows to share the folder structure among different systems (`XCFG_ARCH` overrides the name). An optional first argument ("tagname") changes the name of the build and installation folder. Further arguments are passed to CMake.

References

[https://github.com/susilehtola/libwignernj%5D(https://github.com/susilehtola/libwignernj)
[https://www.cp2k.org/static/downloads/%5D(https://www.cp2k.org/static/downloads/)

LIBXC

To download, configure, build, and install LIBXC for CP2K (LIBXC 7.1 or later), one may proceed as shown below.

```
wget https://www.cp2k.org/static/downloads/libxc-7.1.2.tar.bz2
tar xvf libxc-7.1.2.tar.bz2
cd libxc-7.1.2

wget --content-disposition https://github.com/hfp/xconfigure/raw/main/configure-get.sh
chmod +x configure-get.sh
./configure-get.sh libxc
```

To configure, build, and install using the GNU Compiler Collection (`../libxc/<arch>/gnu`):

```
./configure-libxc.sh
cmake --build build-<arch>-gnu --target install -j $(nproc)
```

To use the Intel Compiler instead (`../libxc/<arch>/icx`):

```
./configure-libxc-icx.sh
cmake --build build-<arch>-icx --target install -j $(nproc)
```

Here, `<arch>` names the target as determined by the compiler (e.g., `sapphirerapids` or `neoverse-v2`), which allows to share the folder structure among different systems (`XCFG_ARCH` overrides the name). The Fortran interface and third-order derivatives (`MAXORDER=3`) are built as needed by CP2K. An optional first argument ("tagname") changes the name of the build and installation folder. Further arguments are passed to CMake.

References

[https://libxc.gitlab.io/%5D(https://libxc.gitlab.io/)
[https://gitlab.com/libxc/libxc%5D(https://gitlab.com/libxc/libxc)

LIBXSMM

LIBXSMM is a library for specialized dense and sparse matrix operations, and deep learning primitives. The build instructions can be found at [https://github.com/libxsmm/libxsmm%5D(https://libxsmm.readthedocs.io/#build-instructions) (PDF).

Appendix

CP2K MPI/OpenMP-hybrid Execution (PSMP)

Overview

CP2K's grid-based calculation as well as DBCSR's block sparse matrix multiplication (Cannon algorithm) prefer a square-number for the total rank-count (2d communication pattern). This is not to be obfuscated with a Power-of-Two (POT) rank-count that usually leads to trivial work distribution (MPI).

It can be more efficient to leave CPU-cores unused to achieve this square-number property rather than using all cores with a "non-preferred" total rank-count (sometimes a frequency upside over an "all-core turbo" emphasizes this property further). Counter-intuitively, even an unbalanced rank-count per node i.e., different rank-counts per socket can be an advantage. Pinning MPI processes and placing threads requires extra care to be taken on a per-node basis to load a dual-socket system in a balanced fashion or to set up space between ranks for the OpenMP threads.

Because of the above-mentioned complexity, a script for planning MPI/OpenMP-hybrid execution (`plan.sh`) is available. Here is a first example for running the PSMP-binary on an SMP-enabled (Hyperthreads) dual-socket system with 24 cores per processor/socket (96 hardware threads in total). At first, a run with 48 ranks and 2 threads per core comes to mind (`48x2`). However, for instance 16 ranks with 6 threads per rank can be better for performance (`16x6`). To easily place the ranks, Intel MPI is used:

```
mpirun -np 16 \
  -genv I_MPI_PIN_DOMAIN=auto -genv I_MPI_PIN_ORDER=bunch \
  -genv OMP_PLACES=threads -genv OMP_PROC_BIND=SPREAD \
  -genv OMP_NUM_THREADS=6 \
  cp2k/<arch>/gnu/bin/cp2k.psmp workload.inp
```

Note: For hybrid codes, `I_MPI_PIN_DOMAIN=auto` is recommended as it spaces the ranks according to the number of OpenMP threads (`OMP_NUM_THREADS`). It is not necessary and not recommended to build a rather complicated `I_MPI_PIN_PROCESSOR_LIST` for hybrid codes (MPI plus OpenMP). To display and to log the pinning and thread affinization at the startup of an application, `I_MPI_DEBUG=4` can be used with no performance penalty. The recommended `I_MPI_PIN_ORDER=bunch` ensures that ranks per node are split as even as possible with respect to sockets (e.g., 36 ranks on a 2x20-core system are put in 2x18 ranks instead of 20+16 ranks).

To achieve a similar placement with OpenMPI, ranks are mapped to "execution slots" (`--map-by slot`) along with specifying the number of processing elements (`PE`). By default, execution slots are counted in number of physical cores which yields `--map-by slot:PE=3` for the same system (mentioned above).

```
mpirun -np 16 --map-by slot:PE=3 \
  -x OMP_PLACES=threads -x OMP_PROC_BIND=SPREAD \
  -x OMP_NUM_THREADS=6 \
  cp2k/<arch>/gnu/bin/cp2k.psmp workload.inp
```

Note: Intel MPI's `I_MPI_PIN_ORDER=bunch` to balance the number of ranks between sockets (see above) appears hard to achieve with OpenMPI therefore an undersubscribed system may not be recommended. To display and to log the pinning and thread affinization at the startup of an application, `mpirun --report-bindings` can be used.

The end of the next section continues with our example and extends execution to multiple nodes of the above-mentioned system.

Plan Script

To configure the plan-script, the metric of the compute-nodes can be given for future invocations so that only the node-count is required as an argument. The script's help output (`-h` or `--help`) initially shows the "system metric" of the computer the script is invoked on. For a system with 48 cores (two sockets, SMP/HT enabled), setting up the "system metric" looks like (`plan.sh <num-nodes> <ncores-per-node> <nthreads-per-core> <nsockets-per-node>`):

```
./plan.sh 1 48 2 2
```

The script is storing the arguments (except for the node-count) as default values for the next plan (file: `$HOME/.xconfigure-cp2k-plan`). This allows us to supply the system-type once, and to plan with varying node-counts in a convenient fashion. Planning for 8 nodes of the above kind yields the following output (`plan.sh 8`):

```
================================================================================
384 cores: 8 node(s) with 2x24 core(s) per node and 2 thread(s) per core
================================================================================
[48x2]: 48 ranks per node with 2 thread(s) per rank (14% penalty)
[24x4]: 24 ranks per node with 4 thread(s) per rank (14% penalty)
[12x8]: 12 ranks per node with 8 thread(s) per rank (33% penalty)
--------------------------------------------------------------------------------
[32x3]: 32 ranks per node with 3 thread(s) per rank (34% penalty) -> 16x16
[18x5]: 18 ranks per node with 5 thread(s) per rank (25% penalty) -> 12x12
[8x12]: 8 ranks per node with 12 thread(s) per rank (0% penalty) -> 8x8
[2x48]: 2 ranks per node with 48 thread(s) per rank (0% penalty) -> 4x4
--------------------------------------------------------------------------------
```

The first group of the output displays POT-style (trivial) MPI/OpenMP configurations (penalty denotes potential communication overhead), however the second group (if present) shows rank/thread combinations with the total rank-count hitting a square number (penalty denotes waste of compute due to not filling each node). For the given example, 8 ranks per node with 12 threads per rank is chosen (`8x12`) and MPI-executed:

```
mpirun -perhost 8 -host node1,node2,node3,node4,node5,node6,node7,node8 \
  -genv I_MPI_PIN_DOMAIN=auto -genv I_MPI_PIN_ORDER=bunch -genv I_MPI_DEBUG=4 \
  -genv OMP_PLACES=threads -genv OMP_PROC_BIND=SPREAD -genv OMP_NUM_THREADS=12 \
  cp2k/<arch>/gnu/bin/cp2k.psmp workload.inp
```

Note: For Intel MPI as well as OpenMPI, mpirun's host-list (`mpirun -host`) is set up with unique node-names, and this is the only style that is explained in this article. There is a competing style where nodes names are duplicated for the sake of enumerating available ranks (or "execution slots" in case of OpenMPI), which is not exercised in this article.

For OpenMPI, the quantity (per node) of the previously mentioned "execution slots" (measured in number of physical cores) are sometimes not known to OpenMPI (depends on cluster/scheduler setup). For instance, `mpirun` may be complaining about an attempt to use too many execution slots simply because OpenMPI believes all systems represent a single such slot (instead of 2x24 cores it only "sees" a single core per system). In such case, it is not recommended to "oversubscribe" the system because rank/thread affinity will likely be wrong (`mpirun --oversubscribe`). Instead, the list of unique nodes names (`-host`) may be augmented with the number of physical cores on each of the nodes (e.g., ":48" in our case).

```
mpirun -npernode 8 -host node1:48,node2:48,node3:48,node4:48,node5:48,node6:48,node7:48,node8:48 \
  --map-by slot:PE=6 --report-bindings \
  -x OMP_PLACES=threads -x OMP_PROC_BIND=SPREAD -x OMP_NUM_THREADS=12 \
  cp2k/<arch>/gnu/bin/cp2k.psmp workload.inp
```

Note: It can be still insufficient to augment the nodes with the expected number of slots (`:48`). If OpenMPI's mpirun is still complaining, it might be caused and solved by the job scheduler. For example, `qsub` (PBS) may be instructed with `-l select=8:mpiprocs=48` in the above case (`mpirun` in this job can use less than 48 ranks per node).

The plan-script also suggests close-by configurations (lower and higher node-counts) that can hit the square-property ("Try also the following node counts"). The example (as exercised above) was to illustrate how the script works, however it can be very helpful when running jobs, especially on CPUs with not many prime factors in the core-count. Remember, the latter can be also the case for virtualized environments that reserve some of the cores to run the Hypervisor i.e., reporting less cores to the Operating System (guest OS) when compared to the physical core-count.

References

[https://github.com/hfp/xconfigure/raw/main/config/cp2k/plan.sh%5D(https://github.com/hfp/xconfigure/raw/main/config/cp2k/plan.sh)
[https://xconfigure.readthedocs.io/cp2k/%5D(https://xconfigure.readthedocs.io/cp2k/)
[https://software.intel.com/content/www/us/en/develop/articles/pinning-simulator-for-intel-mpi-library.html%5D(https://software.intel.com/content/www/us/en/develop/articles/pinning-simulator-for-intel-mpi-library.html)